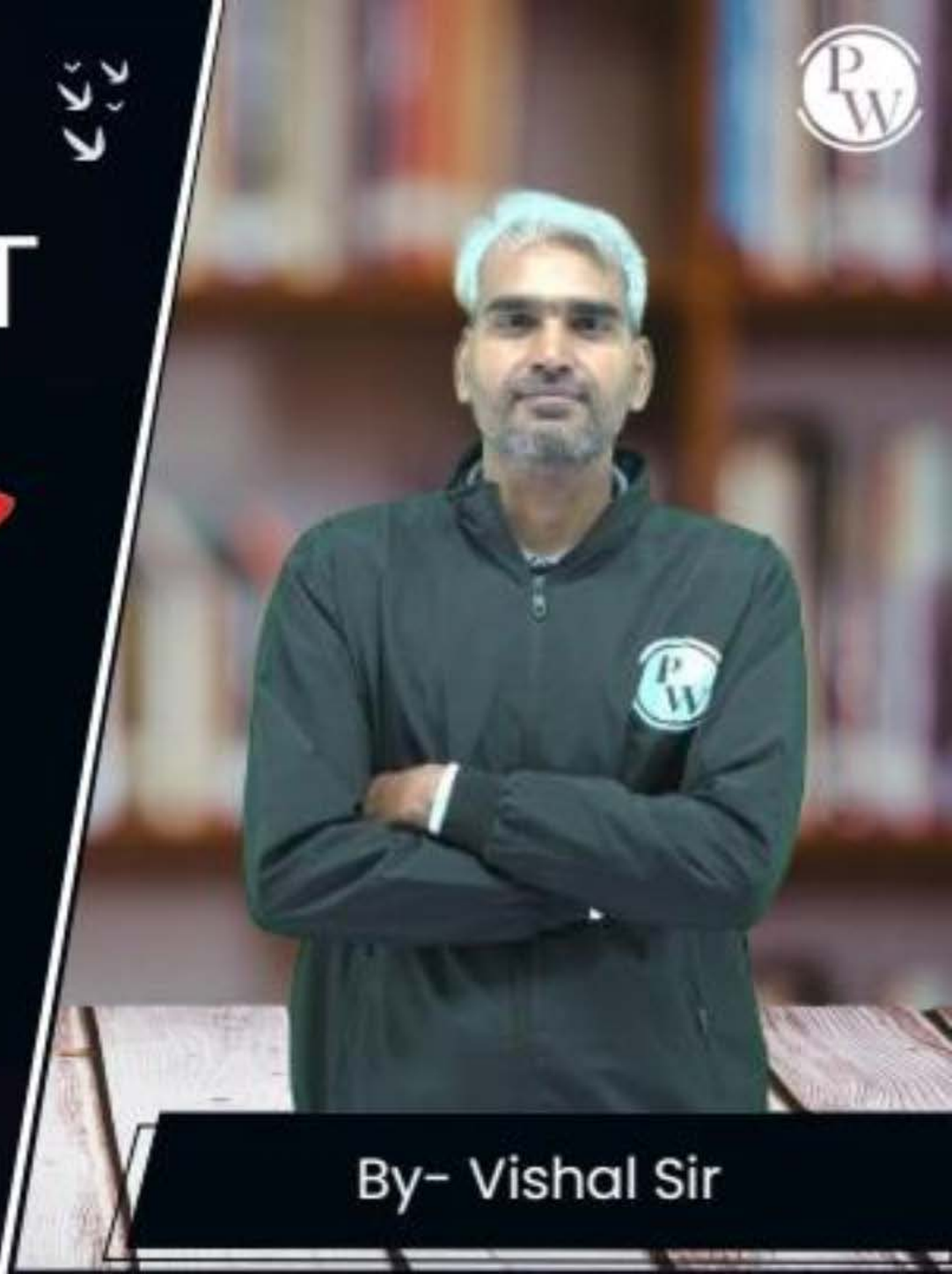
Computer Science & IT



Database Management System

Query Languages

Lecture No. 10



By- Vishal Sir

Recap of Previous Lecture



- Topic: AS clause
- Topic: WITH clause
- Topic: Comparison with NULL

Slide

Topics to be Covered



- Topic: Comparison with regular expression
- Topic: Introduction to relational calculus
- Topic: Tuple relational calculus (TRC)
- Topic: Practice questions

Slide



Topic : Regular expression

'_' :→ Underscore represent exactly one character.

Underscore →

'%' :→ Represent 0 or more (any number) Character

- Write the regular Expression for the names Starting with A

Ans = 'A%'

first alphabet must be 'A'

after that there may be any-thing



Topic : Comparison with Regular expression

To Compare with regular Expression operator used is 'LIKE'

Complement of 'LIKE' is 'NOT LIKE'

EMP

Eid	Ename	Pass_no
E_1	Amit	123
E_2	Anand	379
E_3	Ramesh	NULL
E_4	Raghvendra	239
E_5	Amitabh	386
E_6	Anuj	240

* Retrieve the records of Employee whose name contains exactly 4 letters

```
Select *
    From EMP
        Where Ename LIKE '____'
```

* Retrieve record of the Employee whose name start with R, third letter from the last is 'd' and contain at least 8 letters.

```
Select *
    From EMP
        Where Ename LIKE 'R__%d__'
```

Relational Calculus



Topic : Relational Calculus

- Relational Calculus is a non-procedural query language.
- It uses predicate calculus(First Order Logic) instead of algebra.
  - ∃, ∀, etc.
- Relational calculus provides description about the query to get the result, whereas Relational Algebra gives the method to get the result.



Topic : Types of Relational Calculus

- There are two types of Relational Calculus.

1. Tuple Relational Calculus: Works on tuple (or rows)
2. Domain Relational Calculus: Works on domain of attribute (column)



Topic : Syntax of TRC

it is used to produce the output

Output tuple variable

Set

- Basic syntax of TRC is { t | P(t) }

Predicate formula over tuple variable 't'.

where t is a tuple variable and P(t) is a predicate formula that describes the conditions that must be satisfied by the resulting tuples.

1. It results the set of tuples t, s.t. P(t) is true for t.
2. t ∈ *R*elation denotes that the 't' is a tuple from Relation, also denoted using Relation(t).
3. If 't' is a tuple variable, then "t[A]" / "t.A" specify attribute A in tuple t.
4. P is a predicate formula similar to that of the predicate calculus.



Topic : Equivalence between SQL, RA and TRC

SQL ➤ SELECT distinct * FROM Employee

R.A. ➤ (Employee)

TRC ➤ {t | t ∈ Employee}
or
{t | Employee(t) }

$\{ t \mid t \in \text{Employee} \}$

(or)

Employee (t)

Output tuple Variable

It is our predicate Cond^n

this predicate Formula will return true for all the tuples belonging to relation Employee

∴ O/P will Contain all tuples of Employee



Topic : Free and Bounded variables

- A variable which is preceded by a quantifier is called bounded variable.

- A variable which is not preceded by any quantifier is called free variable.

- There is an important restriction:

  The variable 't' that appears before '|' symbol must be the only free variable in formula P(t).

  In other words all other variables must be bounded using quantifier



Topic : Equivalence between SQL, RA and TRC

SQL ➤ SELECT distinct ename FROM Employee

R.A. ➤ $\pi_{ename}(Employee)$

TRC ➤ { t.ename | t ∈ Employee}

or

{ t[ename] | t ∈ Employee}

or

{ t | ∃S ∈ Employee (t[ename]= S[ename])}

or

{ t | ∃S ∈ Employee (t.ename= S.ename)}

{ t.ename | t ∈ Employee }

{ t[ename] | t ∈ Employee }

{ t | ∃S ∈ Employee (t.ename = S.ename) }

attribute creation w.r.t. tuple 't'

$$\{ t \mid \exists S \in \text{Employee} \wedge t.\text{ename} = S.\text{ename} \}$$

#Q. Retrieve tuples from the relation student where marks are more than 50

Student(sid, name, marks)

① $\{ t \mid t \in Student \wedge t.marks > 50 \}$

(Predicate Condⁿ)

② $\{ t \mid \exists S \in Student (S.marks > 50 \wedge (t.sid = S.Sid \wedge t.name = S.name \wedge t.marks = S.marks)) \}$

#Q. Retrieve sids of all the students from the relation student whose marks are more than 50 .

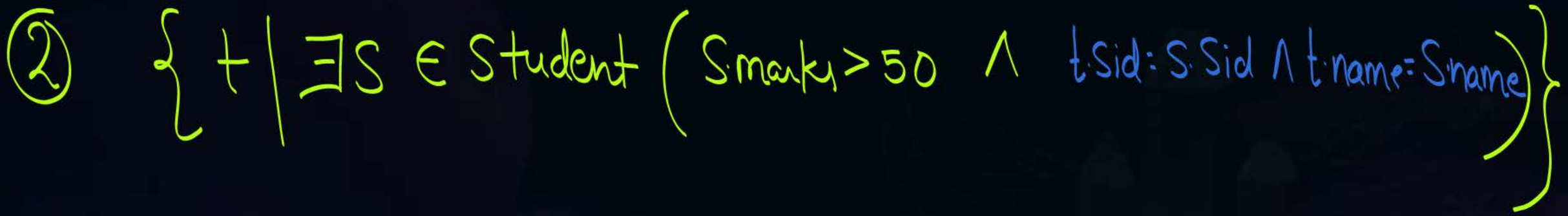
Student(sid, name, marks)

① $\{ t.sid \mid t \in Student \wedge t.marks > 50 \}$

② $\{ t \mid \exists S \in Student (S.marks > 50 \wedge t.sid = S.sid) \}$

#Q. Retrieve sids and names of all the students from the relation student whose marks are more than 50 .
Student(sid, name, marks)





#Q. Consider the following relational schema:

Suppliers(sid, sname, city)

Parts(pid, pname, color)

Catalog(sid, pid, cost)

Find sid of all the suppliers who have supplied part with pid=P1

① $\{ t.Sid \mid t \in Catalog \wedge t.Pid = 'P_1' \}$

② $\{ t \mid \exists C \in Catalog \left(C.Pid = 'P_1' \wedge t.sid = C.Sid \right) \}$

#Q. Consider the following relational schema:

Suppliers(sid, sname, city)

Parts(pid, pname, color)

Catalog(sid, pid, cost)

Find sids of all the suppliers who have supplied some 'red' color parts

① $\{ t.Sid \mid t \in Catalog \wedge \exists P \in Parts \ (t.Pid = P.Pid \wedge P.Color = 'Red') \}$

② $\{ t \mid \exists C \in Catalog \ (\exists P \in Parts \ (C.Pid = P.Pid \wedge P.Color = 'Red') \wedge t.Sid = C.Sid) \}$

③ $\{ t \mid \exists C \in Catalog \ (t.Sid = C.Sid \wedge \exists P \in Parts \ (C.Pid = P.Pid \wedge P.Color = 'Red')) \}$

#Q. Consider the following relational schema:

Suppliers(sid, sname, city)

Parts(pid, pname, color)

Catalog(sid, pid, cost)

[redacted]

Find Sid and Sname of all the suppliers who have supplied some Red Color Parts

$$\{ t.Sid, t.Sname \mid t \in Suppliers \wedge (\exists C \in Catalog ((t.Sid = C.Sid) \wedge \exists P \in Parts (C.Pid = P.Pid \wedge P.Color = 'Red'))) \}$$

#Q. Consider the following relational schema:

Suppliers(sid, sname, city)

✓ Parts(pid, pname, color)

✓ Catalog(sid, pid, cost)

[redacted]

→ Retrieve Sids of the Suppliers who have supplied some Red Color Parts along with the names of those Red Color Parts

$$\{ t \mid \exists C \in Catalog \left(\exists P \in Parts \left(C.Pid = P.Pid \wedge P.Color = 'Red' \right) \right.$$

$$\left. \wedge \; t.Sid = C.Sid \wedge t.pname = P.pname \right) \}$$



Topic : Unsafe TRC query

$\{ t \mid t \notin \text{Relation} \}$

Here we are trying to select all the tuples that does not belong to the Relation

or

$\{ t \mid \neg(t \in \text{Relation}) \}$

tuples belonging to relation

Complement of the tuples belonging to Relation $\Rightarrow$ i.e. tuples not belonging to the Relation

If domain of an attribute happens to be infinite, then such TRC query may produce infinite tuples in output.

Note :-

① Unsafe TRC query may produce infinite number of tuples.

② Given an unsafe TRC query, we can never write an Equivalent Relational algebra Expression

③ For Every safe TRC query we can write an Equivalent Relational Algebra Expression

* Power of TRC (Safe + Unsafe) $\neq$ Power of Relational Algebra

Not Equal because of Unsafe TRC query

Power of Safe TRC = Power of Relational Algebra

H.W.

#Q. Which of the following relational calculus expressions is not safe?

A. $\{t \mid \exists u \in R1\ (t[A] = u[A]) \wedge \neg \exists s \in R2\ (t[A] = s[A]\}$

B. $\{t \mid \forall u \in R1\ (u[A] = \text{“x”} \Rightarrow \exists s \in R2\ (t[A]=s[A] \wedge s[A] = u[A]))\}$

C. $\{t \mid \neg(t \in R1)\}$

D. $\{t \mid \exists u \in R1\ (t[A] = u[A]) \wedge \exists s \in R2\ (t[A] = s[A])\}$

#Q. Consider the relation employee (name, sex, supervisorName) with name as the key. Supervisor Name gives the name of the supervisor of the employee under consideration. What does the following Tuple Relational Calculus query produce?

$\{e.name \mid employee(e) \wedge (\forall x)\ [\neg employee(x) \vee x.supervisorName \neq e.name \vee x.sex = \text{“male”}]\}$

A. Names of employees with a male supervisor.

B. Names of employees with no immediate male subordinates

C. Names of employees with no immediate female subordinates

D. Names of employees with a female supervisor.

#Q. Which of the following tuple relational calculus expression(s) is/are equivalent to $\forall t \in r(P(t))$?

I $\neg\exists t \in r(P(t))$

II $\exists t \notin r(P(t))$

III $\neg\exists t \in r(\neg P(t))$

IV $\neg\exists t \notin r(\neg P(t))$

A. I only

B. II only

C. III only

D. III and IV only

#Q. Consider The Following Relational Scheme

Student (school-id, sch-roll-no, sname, saddress)

School (school-id, sch-name, sch-address, sch-phone)

Enrolment (school-id, sch-roll-no, erollno, examname)

ExamResult (Erollno, examname, marks)

Consider the following tuple relational calculus query

$$\{ t \mid \exists E \in \text{Enrolment}\ t = E.\text{school-id} \wedge$$

$$\mid \{ x \mid x \in \text{Enrolment} \wedge x.\text{school-id} = t \wedge (\exists B \in \text{ExamResult}\ B.\text{erollno} = x.\text{erollno} \wedge$$

$$B.\text{examname} = x.\text{examname} \wedge B.\text{marks} > 35 \} \mid \div$$

$$\mid \{ x \mid x \in \text{Enrolment} \wedge x.\text{school-id} = t \} \mid * 100 > 35 \}$$

If a student needs to score more than 35 marks to pass an exam what does the query return?

#Q.

A. The empty set

B. schools with more than 35% of its students enrolled in some exam or the other

C. schools with a pass percentage above 35% over all exams taken together

D. schools with a pass percentage above 35% over each exam

#Q. Consider a database that has the relation schemas
EMP(EmpId, EmpName, DeptId) and
DEPT(DeptName, DeptId).
Note that the DeptId can be permitted to a NULL in the relation EMP. Consider the following queries on the database expressed in tuple relational calculus.

I. $\{t \mid \exists u \in EMP\ (t[EMPName] = u[EmpName] \wedge \forall v \in DEPT\ (t[DeptId] \neq DeptId]))\}$

II. $\{t \mid \exists u \in EMP\ (t[EMPName] = u[EmpName] \wedge \exists v \in DEPT\ (t[DeptId] \neq DeptId]))\}$

III. $\{t \mid \exists u \in EMP\ (t[EMPName] = u[EmpName] \wedge \exists v \in DEPT\ (t[DeptId] = DeptId]))\}$

Which of the above queries are safe?

A. I and II only
B. I and III only
C. II and III only
D. I, II and III



2 mins Summary

- **Topic** Comparison with regular expression
- **Topic** Introduction to relational calculus
- **Topic** Tuple relational calculus
- **Topic** Practice questions

THANK - YOU